IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

FraudOps-Bench: Process-Level, Tool-Grounded Evaluation of LLM Agents for Card-Not-Present Fraud Investigation, and a Calibration Failure Mode Worth Knowing About

AYUSHI AMBILKAR[1] AND PRAMEGH UIKEY[2]
[1]Independent Researcher, India (e-mail: ambilkarayushi@gmail.com)
[2]Independent Researcher, India (e-mail: uikeyprameghaa@gmail.com)

Corresponding author: Ayushi Ambilkar (e-mail: ambilkarayushi@gmail.com).

ABSTRACT We study whether a large language model (LLM) agent can emulate a fraud analyst's card-not-present transaction review workflow – not just predict a fraud label, but gather evidence via tools, reason under an explicit standard operating procedure (SOP), and produce an auditable disposition with cited evidence. We build FraudOps-Bench: a six-tool investigation environment over IEEE-CIS transaction data, a LangGraph-based agent (brain/orchestrator/tools/memory/supervisor), and three comparison arms (a zero-evidence control, a single-shot "all evidence at once" baseline, and the full iterative agent) alongside a classical gradient-boosted-tree baseline. Using a selective-prediction framework (escalate-to-human on low-confidence cases), we initially found the agentic arm competitive with classical ML on a 120-case calibration set. That result did not survive a fresh 300-case holdout: the calibration procedure had overfit small-sample noise in the escalation threshold. We diagnose the failure precisely – discretized, round-number-anchored LLM probability outputs interacting badly with a narrow band selected on too little data – fix it with a lower-confidence-bound selection criterion, and validate the fix on a second, disjoint holdout set. Under the corrected calibration, the agentic arm's accuracy claim holds up (96.1%, its best result on any split) but its practical coverage collapses to 17% of the queue; the classical baseline remains the strongest *practical* performer, dominating the full risk-coverage curve (area under the risk-coverage curve, AURC, 0.034 vs. 0.108–0.112). We report faithfulness scoring (two independent methods, $\sim$99% evidence-attribution accuracy for both LLM arms), a structured error taxonomy, and an ablation of the agent's self-consistency mechanism. We argue the calibration failure mode itself – not just the fix – is a transferable lesson for anyone doing selective prediction with LLM confidence scores on small calibration sets.

INDEX TERMS agentic artificial intelligence; large language models; fraud detection; selective prediction; risk-coverage analysis; tool-augmented LLMs; model calibration; benchmark evaluation

1. INTRODUCTION

Fraud remains a large and continuous operational burden on financial institutions. The Association of Certified Fraud Examiners' 2024 *Report to the Nations* documents thousands of occupational-fraud cases spanning 138 countries and territories [1], and UK Finance's own H1 2025 figures report 2.09 million confirmed fraud cases and £629.3 million stolen in just six months [2] – a reminder that the review queues behind these numbers are large, continuous, and staffed by human analysts working under time pressure.

Large language models (LLMs) are an obvious candidate to help with this queue, and a growing body of recent work has tested whether an LLM can classify a transaction as fraudulent as well as a classical model trained on standard public benchmarks such as IEEE-CIS [3]. That question is close to saturated: public fraud datasets are anonymized and feature-engineered for tabular models rather than investigative reasoning, and the strongest recent public-data-only

result along these lines, FinFRE-RAG [4], still frames the task as transaction-level risk scoring rather than the workflow a human fraud analyst actually performs.

That workflow looks different from classification. A real analyst does not receive a fully assembled feature vector; they receive an alert, then decide what evidence to pull – card history, device history, prior velocity, identity-match signals – before reasoning under an institution's standard operating procedure (SOP) toward a disposition: approve, escalate, or reject, with a citable rationale. Emulating that *process*, not just its final label, is the harder and less-studied problem. The closest published attempt is the FAA framework [5], which wraps GPT-4o's Assistants application programming interface (API) around a multi-agent investigation pipeline and reports very strong F1 score (the harmonic mean of precision and recall) of 0.98–0.99 on synthetic Sparkov/CCTD data – but on a limited, mostly synthetic sample, without a process-level evaluation of when the system should defer to a human rather than commit to a label. The nearest behavioral analogues to a tool-grounded, policy-constrained investigation agent come not from fraud but from security operations: CORTEX [6] and SIABench [7] both evaluate multi-agent, tool-using triage over auditable evidence in a security operations center (SOC) setting, but on non-public data and in a different operational domain. General-purpose agent benchmarks – SOP-Bench [8], τ-bench [9], IntellAgent [10] – supply the methodological template for evaluating long-horizon, policy-heavy, tool-using agents, but none target fraud operations specifically.

No existing public work, to our knowledge, combines (a) public fraud data, (b) a tool-grounded agent that gathers its own evidence rather than receiving it pre-assembled, and (c) a process-level, auditable evaluation that asks not just "was the label right" but "should this system have committed to a label at all." That is the gap FraudOps-Bench targets.

Building and evaluating such a system surfaced a second, independently interesting result. Selective prediction – escalating low-confidence cases to a human rather than forcing a decision – is the natural evaluation frame for this kind of agent, since a fraud-review system that is wrong with full confidence is far more costly than one that is honest about uncertainty. Doing this well requires calibrating a decision threshold against a labeled set, and we found, by first getting it wrong, that LLM-generated confidence scores are not the well-behaved continuous outputs classical calibration methods assume: they cluster heavily on round, prompt-anchored values, and a threshold selected against a small calibration set can badly overfit that clustering in a way that only shows up on a much larger, independent holdout. We diagnose this failure precisely, fix it, and validate the fix on a second, disjoint holdout set – and we report the failure mode itself as a first-class contribution, since it is a risk any selective-prediction pipeline built on LLM confidence scores over a modestly sized calibration set is likely to share.

Contributions:

1) FraudOps-Bench: a public-data (IEEE-CIS), tool-grounded, LangGraph-based fraud-investigation environment with an explicit SOP, four comparison arms, and a disciplined dev/calibration/holdout evaluation protocol with a frozen-methodology audit mechanism preventing evaluation leakage.
2) A documented case study of calibrate_band() overfitting on a small calibration set – diagnosed via probability-output discretization, fixed with a lower-confidence-bound + minimum-sample-size selection rule, and validated end-to-end on a second, never-touched holdout set.
3) Two independent faithfulness-evaluation methods (deterministic numeric cross-referencing and an LLM-judge pass) plus a structured error taxonomy, going beyond outcome accuracy to ask whether the agent's cited reasoning is actually grounded in the evidence it retrieved.
4) An ablation isolating the contribution of a self-consistency mechanism to the agent's calibrated probability estimates.

The rest of the paper proceeds as follows. Section 2 positions this work against prior fraud-LLM and agent-benchmark literature. Section 3 describes FraudOps-Bench's design. Section 4 reports a two-method faithfulness evaluation of the LLM arms' cited evidence. Section 5 is the central empirical contribution: the calibration failure, its diagnosis, fix, and validation. Section 6 reports a self-consistency ablation. Section 7 reports cost and latency. Section 8 discusses implications, Section 9 states limitations directly, and Section 10 concludes.

2. RELATED WORK

Fraud-specific LLM work. The Fraud Dataset Benchmark [3] established the standard public-data infrastructure this line of work builds on, standardizing IEEE-CIS to a 561,013/28,527 train/test split over 67 retained features and explicitly cataloguing the limitations of public fraud data – sparse identity fields, no case notes, no institutional history – that any LLM system built on it inherits. Within that constraint, FinFRE-RAG [4] is the cleanest recent demonstration that public-data-only LLM fraud scoring can be substantially improved by retrieval and feature reduction, but it remains transaction-level risk scoring, not investigative workflow: the model receives a feature vector, not an alert it must build a case around.

The closest direct prior art to FraudOps-Bench is the FAA framework [5], which wraps GPT-4o's Assistants API around a multi-agent pipeline – planning, evidence gathering, analysis, and report generation, with an optional vision agent – and reports very strong F1 (0.9801 on Sparkov, 0.99 on CCTD) across 500 evaluated transactions, with investigations averaging 5.5–7 steps. This is the paper closest in spirit to "an LLM agent investigates a fraud case," but it evaluates on a limited, mostly synthetic sample and does not report a process-level or selective-prediction evaluation – there is no

notion, in that evaluation, of the system declining to commit to a disposition.

Behavioral analogues outside fraud. The strongest evidence that tool-grounded, multi-agent decomposition helps on auditable, high-stakes investigation work comes not from fraud but from security operations. CORTEX [6] shows a multi-agent SOC-triage system improving actionable F1 from 0.66 to 0.78 and cutting false-positive rate from 24.9% to 14.2% over a single-agent baseline, on real production workflow traces. SIABench [7] evaluates frontier models on deep, tool-executing security-incident-analysis workflows and finds they still fail a large fraction of long-context scenarios (GPT-4o failed 11 of 25 in one setting) – direct evidence that process-level, tool-grounded evaluation surfaces failure modes that outcome-only scoring hides. Both operate on non-public data in a different operational domain from fraud, but both are closer behavioral analogues to what FraudOps-Bench measures than any fraud-specific paper we found.

Agent-evaluation methodology. SOP-Bench [8], τ-bench [9], and IntellAgent [10] supply the methodological template this paper adapts to fraud operations: long-horizon, policy-constrained, tool-using agent evaluation, as distinct from single-turn question answering. SOP-Bench in particular shows that larger tool registries can hurt task success (37% vs. 20.8% task-success rate in one ablation) – a caution directly relevant to FraudOps-Bench's six-tool design. The Berkeley Function-Calling Leaderboard [11] evaluates the narrower function/tool-calling capability these systems depend on. FraudOps-Bench's agentic arm is implemented on LangGraph [15], one of several general-purpose multi-agent orchestration frameworks in current use alongside AutoGen [14] and CrewAI [16]; none of these frameworks target fraud operations specifically, and none ship an evaluation protocol beyond their own tool-calling correctness.

Adjacent public fraud datasets. BAF [12] and AML-world [13] are the strongest synthetic-from-real alternatives to IEEE-CIS for future extensions of this benchmark – BAF in particular would extend FraudOps-Bench to account-opening fraud, a structurally different investigation shape from the card-not-present setting studied here (Section 10).

Positioning. Distinguishing this paper from the saturated "LLM vs. classical ML on IEEE-CIS" classification literature [3], [4] is deliberate: FraudOps-Bench's contribution is not a new state-of-the-art classification number, but a process-level, tool-grounded evaluation protocol – including an honest selective-prediction failure mode and its fix – applied to a domain that, to our knowledge, no existing public benchmark combines with a genuinely tool-grounded, multi-step investigation agent.

3. FRAUDOPS-BENCH DESIGN

3.1 TASK AND DATA

Card-not-present transaction risk review over the IEEE Computational Intelligence Society's (IEEE-CIS) Kaggle fraud data [17], the same underlying dataset the Fraud Dataset Benchmark [3] uses as its richest public-data testbed; FraudOps-Bench draws its own dev/calibration/holdout splits directly from the raw competition data (Section 3.4) rather than reusing FDB's fixed train/test split, since our evaluation needs disjoint labeled subsets for calibration and repeated holdout use rather than a single fixed split. Each case exposes a visible alert summary (amount, product code, card type, purchaser/recipient email domain, device type) and, for evidence-bearing arms, access to six tools mirroring what a real fraud analyst's case-management tooling would expose: get_transaction_details, get_card_history, get_email_domain_profile, get_device_history, get_velocity_summary, get_identity_match_summary. Tool outputs are leakage-free by construction (only information available strictly before the current transaction's timestamp).

3.2 COMPARISON ARMS

Arm	Description
direct_control	Sees only the visible alert summary; no tool access. Zero-evidence control.
linear_api	Single-shot completion given the full output of all 6 tools at once.
agentic_api	A LangGraph agent with explicit brain (LLM), orchestrator, tool nodes, structured memory, and a supervisor node that checks required-check completion before allowing finalization. Iterative: the agent chooses which tools to call and when.
classical_ml	HistGradientBoostingClassifier on leakage-free tabular features derived from the same underlying data (no LLM).

All LLM arms use Claude Sonnet 5. classical_ml and the local-model variants (linear_local/agentic_local, built but not run – see Limitations) are not part of this report's headline comparisons beyond classical_ml.

3.3 SELECTIVE PREDICTION AND THE SOP

Every LLM arm outputs a calibrated fraud_probability. Disposition (APPROVE/ESCALATE/REJECT) is a deterministic function of that probability against a per-arm band, not a free-form model choice – the standard selective-prediction (Chow's rule [18]) risk-coverage framing: fix an acceptable error rate on decided cases, escalate the rest. The SOP explicitly instructs the model on a calibration scale (0.0–0.15 no signal, 0.35–0.65 genuinely mixed evidence, 0.85–1.0 strong convergent signal) and requires an explicit per-check verdict (protective/risk/ neutral) for six required checks before a final probability, to counter base-rate-neglect and encourage genuine escalation on ambiguous cases rather than false confidence.

3.4 EVALUATION PROTOCOL: DEV / CALIBRATION / HOLDOUT, WITH A FROZEN-METHODOLOGY GUARD

To prevent the eval-leakage failure mode common in iterative prompt engineering (tuning a prompt or threshold by repeatedly checking accuracy on the set being reported), FraudOps-Bench uses three disjoint splits:

- **dev** ($n = 50$, 25/25 fraud): free to use for iterative SOP/prompt development. Already spent this way; never reported as a final number.
- **calibration** ($n = 120$, 60/60 fraud): used only to select each arm's escalate-probability band via k-fold cross-validation.
- **holdout** ($n = 300$, 150/150 fraud, two independent draws used in this work – see Section 5): single-use final reporting set.

A methodology-freeze mechanism (freeze_methodology.py) hashes every file that defines an arm's behavior (SOP text, model config, prompt-construction code, agent graph definition, calibration logic) into a manifest. The holdout-execution path refuses to run unless the current files match the frozen hashes, with a loud, logged --force override available but never silent. This is the mechanism that makes the before/after account in Section 5 possible: it is provable, not just claimed, that nothing was retuned against the second holdout after the fix was frozen.

4. FAITHFULNESS EVALUATION

Two independent methods, both described in docs/methodology_log.md's 2026-08-14 and 2026-08-18 entries, are used to check whether an arm's cited reasoning is actually grounded in the evidence it retrieved.

Method 1 – deterministic numeric cross-referencing. Extracts every numeric claim from an arm's cited evidence (evidence_used, check_verdicts, risk_indicators, etc.) and checks it against the real tool-evidence values (including a percentage-form match for fractional fields). Zero additional API cost. Across dev, calibration, holdout, and holdout_v2, both LLM arms show 98.8–99.3% verified rates. Residual "unverified" claims are overwhelmingly the model correctly performing arithmetic (e.g. a time delta between two timestamps) that the ground-truth flattener does not itself compute – not fabrication (manually verified, see docs/methodology_log.md).

Method 2 – LLM-judge semantic check. A Haiku-based judge (25-case samples per arm on holdout_v2) checks for semantic misattribution the numeric method cannot catch: a correct number attached to a false claim. Found genuine (if minor) errors in 6/24 (linear_api) and 8/24 (agentic_api) sampled cases – see docs/error_taxonomy.md for the full categorization, including one category (rounding-precision framing) that is arguably not a real error but a limitation of the judge's own literalness, disclosed rather than hidden.

Result: both arms are highly faithful to the evidence they actually retrieve. The accuracy differences reported in Section 5 are not explained by evidence fabrication.

5. THE CALIBRATION FAILURE, DIAGNOSIS, FIX, AND VALIDATION

This section is presented as a first-class result, not an appendix correction – the failure mode is general to selective prediction with LLM-generated probabilities and worth other practitioners knowing about.

5.1 INITIAL RESULT (CALIBRATION SPLIT, N=120)

Band selection via 5-fold cross-validated accuracy on the calibration split initially favored a very narrow escalate band for agentic_api, (0.45, 0.55), yielding 85.7% accuracy at 99% coverage – appearing competitive with classical_ml (86.0% accuracy, 95% coverage).

5.1A REPEAT-RUN VARIANCE (STOCHASTICITY CHECK)

Before diagnosing the holdout regression (Section 5.2) as a calibration problem, we first checked whether it could instead be explained by ordinary generation-time stochasticity. Each API arm was re-run twice on the unchanged $n = 120$ calibration split (rep1, rep2, identical prompts/config, re-evaluated under the same band):

Arm	Run accuracies	Mean	Std
linear_api	0.912, 0.905, 0.880	0.899	±0.017
agentic_api	0.938, 0.969, 0.966	0.957	±0.017

Run-to-run variance from stochasticity alone is small ($\sim\pm$ 1.7 points for both arms) – much smaller than the sampling-noise confidence interval (CI) at $n = 120$ ($\sim \pm$ 6pt) or even $n = 300$ ($\sim \pm$ 4pt, Section 5.5). Single runs are not being meaningfully misled by generation-time randomness; whatever explains the $\sim$10-point holdout regression in Section 5.2 has to be something other than ordinary model stochasticity. This measurement (referenced again in Section 6) is the closest available estimate of single-run noise for both LLM arms, and directly motivates ruling out stochasticity before looking for – and finding – a selection-procedure explanation instead.

5.2 FIRST HOLDOUT (N=300) AND THE REGRESSION

Under the frozen (0.45, 0.55) band, agentic_api scored only 75.9% accuracy on a fresh 300-case holdout – a $\sim$10-point drop. classical_ml held steady (87.3%).

5.3 ROOT CAUSE

agentic_api's fraud_probability outputs are not continuous: they cluster heavily on round 0.05-increment values, with 0.45 and 0.55 – exactly the selected band's edges – the two single most common outputs (53/300 and 45/300 cases on the holdout, respectively). On the 120-case calibration split, only 12 and 13 cases landed at those exact values, and by sampling luck those small groups skewed hard toward one class (75% non-fraud at 0.45, 61.5% fraud at 0.55), making the narrow band look excellent. On the larger holdout, the same exact values are close to coin flips (47.2% and 55.6% fraud respectively) – consistent, in fact, with what the SOP's own calibration-scale instructions say those values should mean (genuinely mixed evidence). The calibration procedure mistook small-sample noise for signal.

5.4 FIX

calibrate_band() was changed from selecting the narrowest band whose mean cross-validated accuracy cleared a target threshold, to selecting the narrowest band whose **lower-confidence-bound (LCB)** (mean minus 1.645 standard errors across folds) clears the threshold, and additionally requiring a minimum total decided-case count across folds regardless of how good a band's point estimate looks. Re-run on the same calibration data, this correctly rejects the narrow band (LCB 0.794 vs. an 0.85 target) and selects (0.25, 0.75) instead (LCB 0.895).

5.5 VALIDATION ON A SECOND, INDEPENDENT HOLDOUT (N=300)

A fresh, disjoint 300-case holdout (holdout_v2, excluded from the classical-model training set along with every prior split) was generated and run under the corrected, re-frozen methodology.

TABLE 1. holdout_v2 results (n=300).

Arm	Band	Cov.	Acc.	Prec.	Rec.	F1	Cov.-wt.
linear_api	(.35,.65)	50.7%	0.914	0.917	0.946	0.931	46.3%
agentic_api	(.25,.75)	17.0%	0.961	0.944	1.000	0.971	16.3%
classical_ml	(.35,.65)	85.3%	0.910	0.876	0.971	0.921	77.7%
direct_control	(.4,.6) def.	0%	–	–	–	–	–

"Coverage-weighted" = coverage × accuracy, the fraction of the full queue resolved correctly – the fairer single-number comparison when arms escalate at very different rates.

Table 1b: same three arms, matched at all three notable bands. Table 1 compares each arm at its own independently-selected operating point, which is the correct primary comparison but obscures how each arm performs off that point. Since disposition is a deterministic function of fraud_probability and the band, every arm's accuracy/coverage at *any* band is a free re-computation on the already-collected holdout_v2 probabilities (Section 5.5's Figure 1 curve) – no new API calls. Shown here for all three arms at the three bands discussed in this paper: the original overfit band, the common band linear_api/classical_ml were independently selected at, and the wide band agentic_api was independently selected at. Each arm's own selected operating point (the one reported in Table 1) is **bolded**.

TABLE 1b. All three arms at all three notable bands.

Arm	(.45,.55)	(.35,.65)	(.25,.75)
linear_api	cov 96.7%, acc 0.752	**cov 50.7%, acc 0.914**	cov 37.3%, acc 0.929
agentic_api	cov 99.7%, acc 0.759	cov 39.7%, acc 0.924	**cov 17.0%, acc 0.961**
classical_ml	cov 95.3%, acc 0.888	**cov 85.3%, acc 0.910**	cov 73.7%, acc 0.937

Where each method shines, read directly off this table:

- **classical_ml's advantage is holding accuracy while staying willing to decide.** At the tightest band it still resolves 95.3% of the queue at 88.8% accuracy – both other arms are 20+ points less accurate at that same near-total coverage (0.752, 0.759). This is the source of its risk-coverage-curve dominance (Section 5.5, Figure 1): it doesn't trade coverage for accuracy nearly as steeply as the LLM arms do.
- **agentic_api's advantage is ceiling accuracy at its most selective point.** At the wide (0.25, 0.75) band, it is the single most accurate cell in this entire table (0.961) – edging out classical_ml's 0.937 at the same band – but at less than a quarter of classical_ml's coverage (17.0% vs. 73.7%). When agentic_api commits, its confidence is well-placed; it simply commits rarely.
- **linear_api never wins a cell outright** but is never far behind either, sitting between the other two on both axes at every band – consistent with its middling coverage-weighted score in Table 1.
- The practical reading: classical_ml is the stronger choice as a primary decision-maker across the board; agentic_api's comparative advantage is real but narrow – a high-precision second opinion on the small slice of cases it is willing to rule on, not a general-purpose replacement.

TABLE 2. Bootstrap 95% CIs (10,000 resamples, src/stats_analysis.py).

Arm	Accuracy		Coverage		Cov.-wt.	
linear_api	0.915	[0.868, 0.957]	0.507	[0.450, 0.563]	0.463	[0.407, 0.520]
agentic_api	0.961	[0.900, 1.000]	0.170	[0.130, 0.213]	0.164	[0.123, 0.207]
classical_ml	0.910	[0.873, 0.944]	0.853	[0.813, 0.893]	0.777	[0.727, 0.823]

TABLE 3. Pairwise McNemar's tests (paired, restricted to cases both arms decided).

Comparison	n (both)	b	c	p
linear_api vs. agentic_api	50	0	0	1.0000
linear_api vs. classical_ml	137	4	5	1.0000
agentic_api vs. classical_ml	47	0	2	0.5000

Because arms with very different coverage (17% vs. 85%) share few commonly-decided cases, these paired tests are underpowered by construction (see outputs/holdout_v2/stats_report.md) and are reported for completeness rather than as the primary evidence – the bootstrap CIs and risk-coverage curves above are more informative when coverage differs this much.

5.6 INTERPRETATION

The agentic_api accuracy claim is real. It holds up on completely fresh data under a properly calibrated band – 96.1%, its best result on any split – confirming the original problem was the band, not a capability regression. But the honest cost

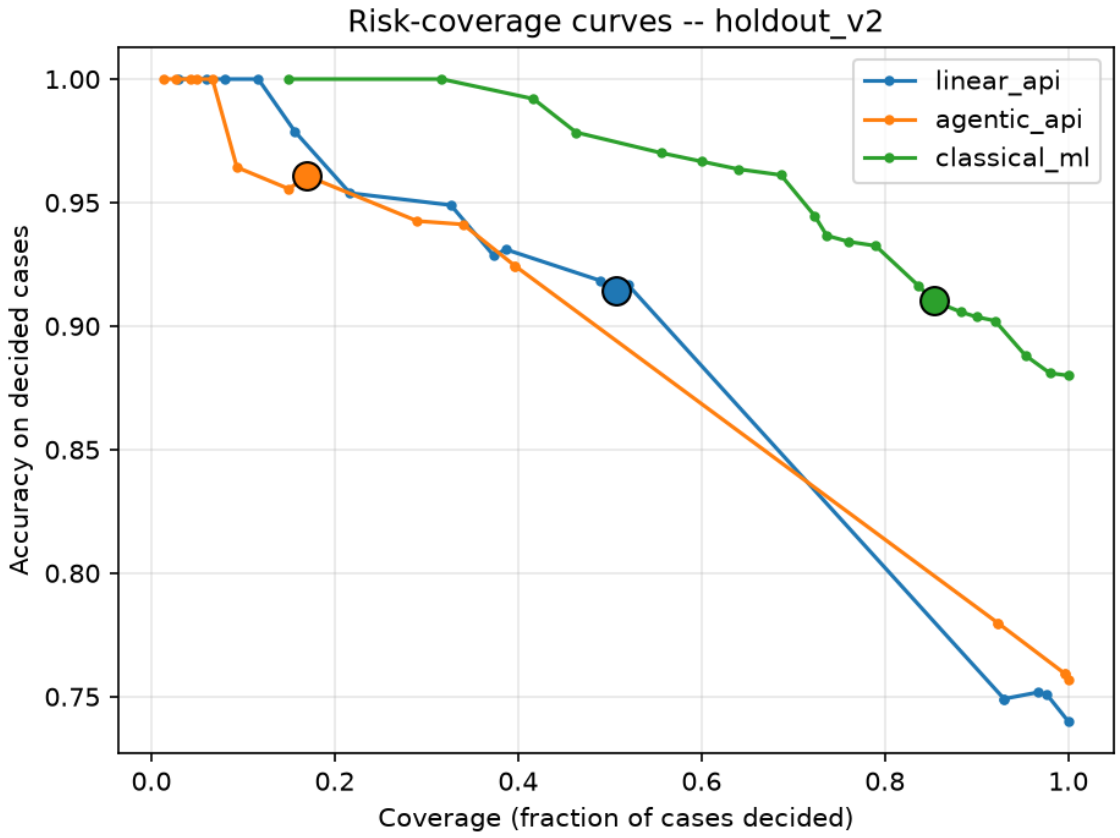

FIGURE 1. Full risk-coverage curves per arm (outputs/holdout_v2/risk_coverage_curves.png), with the actual reported operating point marked. classical_ml's curve dominates both LLM arms across nearly the entire coverage range, not merely at the specific chosen operating points – area under the risk-coverage curve (AURC) 0.0343 (classical_ml) vs. 0.1080 (linear_api) vs. 0.1117 (agentic_api); lower is better.

is now visible: at that accuracy, it escalates 83% of the queue. classical_ml remains the strongest *practical* performer by a wide margin, both by coverage-weighted correctness (77.7% vs. 16.3%) and by dominating the full risk-coverage curve, not just the single chosen point. linear_api sits between the two on both axes, as it did on every split tested.

6. ABLATION: SELF-CONSISTENCY

Self-consistency (resolve_with_second_opinion, src/selective_prediction.py) draws a second independent probability estimate whenever the first lands inside the escalate band, and nudges the estimate past the band edge on agreement – intended to convert genuine near-misses into confident decisions without forcing a guess on true toss-ups. To test whether this mechanism actually changes outcomes, linear_api and agentic_api were re-run on the identical 300 holdout_v2 cases with --override-self-consistency false, under the same frozen bands used for the primary report ((0.35, 0.65) and (0.25, 0.75) respectively) – an architecture-vs-architecture comparison on a fixed decision threshold, not a re-tune.

TABLE 4. Self-consistency on vs. off, holdout_v2 (n=300).

Arm	Condition	Coverage	Accuracy
linear_api	on (primary)	50.7%	0.914
linear_api	off (ablation)	51.7%	0.903
agentic_api	on (primary)	17.0%	0.961
agentic_api	off (ablation)	16.3%	0.980

McNemar's test, restricted to cases decided under both conditions: linear_api ($n = 135$ both-decided): 0 discordant pairs, $p = 1.0$. agentic_api ($n = 40$ both-decided): 0 discordant pairs, $p = 1.0$.

Result: no detectable effect. Neither coverage nor accuracy moves by more than calibration-split single-run stochasticity (Section 5.1a: ± 1.7 points for both LLM arms, measured via repeat runs on the $n = 120$ calibration set – the closest available estimate of run-to-run noise, though not measured directly on holdout_v2), and on every case decided under both conditions, self-consistency changed the correctness of exactly zero of them. As implemented, self-consistency is a mechanistically plausible but empirically inert intervention for this task and model at this operating point – a negative result reported directly rather than selectively omitted. It also confirms the accuracy differences reported in

Section 5 are attributable to the calibration-band fix, not to self-consistency's presence.

Incidental finding: self-consistency's design intent (spend extra calls only on genuinely ambiguous cases) is corroborated by cost: turning it off saved $10.04 (linear_api) and $6.85 (agentic_api) across 300 cases each – the mechanism is triggering roughly as often as expected given the bands, even though it isn't moving the accuracy needle at this operating point.

7. COST AND LATENCY

Arm	Mean cost/case	Mean latency/case
direct_control	$0.047	29.7s
linear_api	$0.130	64.3s
agentic_api	$0.132	57.8s
classical_ml	∼$0 (local)	∼0s

Full holdout_v2 run ($n = 300$, 3 API arms): $92.84, zero errors across 900 successful API calls (one transient truncation retried successfully under the unchanged frozen configuration, consistent with a 0.33% base rate for this failure mode observed across both holdout rounds). Self-consistency ablation (Section 6, both arms, same 300 cases, self-consistency off): $61.87, confirming the mechanism's cost is concentrated on the intended ambiguous-case subset.

8. DISCUSSION

The calibration-band failure documented in Section 5 is not specific to fraud or to this benchmark. Any pipeline that (a) elicits a numeric confidence score from an LLM and (b) calibrates a decision threshold against a modestly sized labeled set should expect the same failure mode: LLM probability outputs are not the smooth, continuous quantities classical calibration methods assume – they cluster on round, prompt-anchored values (Section 5.3), and a threshold-selection procedure that optimizes a point estimate of accuracy, rather than a variance-aware lower bound, can mistake small-sample class imbalance within one of those clusters for genuine signal. The fix used here – select on an LCB criterion (Section 5.4) rather than a point estimate, and enforce a minimum decided-case count per candidate band – is a general prescription for this class of problem, not a fraud-specific patch, and Section 5.1a's stochasticity check ($\pm$1.7pt run-to-run noise, far smaller than the $\sim$10-point holdout regression this failure produced) is what let us rule out ordinary generation randomness as the explanation before looking for – and finding – a selection-procedure bug instead.

This failure mode also has a methodological implication beyond its fix: selective prediction changes which cross-arm comparison is meaningful. A single point-accuracy number is not a fair comparison between arms that escalate at very different rates – agentic_api at 17% coverage and classical_ml at 85% coverage are not competing on the same axis, and Table 1b's per-band comparison and Section 5.5's risk-coverage curves (Figure 1) are the correct way to read the results: classical_ml dominates the full curve (AURC 0.0343 vs. 0.108–0.112), not merely the specific operating points each arm happened to select.

Faithfulness and outcome accuracy are also separable results, not the same finding read two ways. Both LLM arms are highly faithful to the evidence they retrieve ($\sim$99% deterministic verified rate, Section 4), and the residual genuine errors caught by the LLM-judge pass are concentrated in minor evidence-handling categories (miscounting, cross-record misattribution) rather than fabrication (docs/error_taxonomy.md). The most common source of an outright wrong disposition is error-taxonomy category 9 – cases where every cited fact is correct and the reasoning is coherent, but the case itself was genuinely hard (e.g. HOLD2_0104: a well-corroborated multi-signal false positive). That is ordinary task difficulty under real uncertainty, not evidence fabrication, and it is worth stating plainly: a system can be trustworthy in how it uses evidence while still being wrong on the cases that are actually hard, and conflating the two would misdiagnose where to invest further effort (better retrieval and grounding vs. better judgment on ambiguous cases).

Put together, the practical reading for a fraud-ops team evaluating this class of system is that classical ML remains the stronger default: it resolves far more of the queue at comparable or better accuracy (Table 1, coverage-weighted 77.7% vs. 16.3%) and dominates the full risk-coverage trade-off. The LLM agent's demonstrated value in this evaluation is a high-precision, low-coverage second opinion – its single most accurate operating point in this study (96.1%, agentic_api at its selected band) edges out classical ML's accuracy at the same band, but at less than a quarter of the coverage – not a general-purpose replacement for the classical model.

ETHICAL CONSIDERATIONS

FraudOps-Bench uses only the public, already-anonymized IEEE-CIS Kaggle competition dataset [17] – no real customer personally identifiable information (PII), case notes, or institutional data were used or generated. The fraud-analyst SOP driving all LLM arms' reasoning was authored by the researchers for this benchmark and has not been validated against a practicing analyst's judgment (Section 9); it should not be read as a claim about real institutional practice. No fairness or demographic-parity audit was performed on any arm's dispositions, and IEEE-CIS provides no demographic attributes to audit against directly – a limitation worth flagging explicitly, since a fraud-decisioning system's false-positive and false-negative costs are not symmetric across a real customer population, even though that asymmetry is outside what this dataset can measure. This work reports a research benchmark, not a deployed or deployment-ready system, and none of its dispositions were used to make a real decision about a real transaction or person.

9. LIMITATIONS

Stated directly, not hedged:

- **Single dataset.** IEEE-CIS only. No cross-dataset generalization claim is made or supported by this work.
- **Single model.** Claude Sonnet 5 only. No cross-model comparison (e.g. GPT-4o, Gemini) was run.
- **Self-authored SOP.** The fraud-analyst SOP was written by the researchers, not validated by a practicing fraud analyst. It may not reflect real institutional practice.
- **Ground truth.** IEEE-CIS isFraud labels are treated as ground truth without independent verification; known label-noise concerns in Kaggle competition data are not specifically audited here.
- **No human baseline.** This work reports agent-vs.-classical-ML comparisons only; no claim is made about matching or exceeding human analyst performance.
- **Calibration-set size relative to output discretization.** The root-cause finding in Section 5 is itself evidence that a 120-case calibration set is marginal for this class of method when the underlying model's probability outputs are discretized. The fix (LCB-based selection) mitigates but does not eliminate this risk; a larger calibration set would be a more robust long-term fix.
- **McNemar's tests underpowered.** Section 5.5's paired significance tests have low power given how different the arms' coverage profiles are; the bootstrap CIs and risk-coverage curves carry more of the evidentiary weight in this work.

10. CONCLUSION AND FUTURE WORK

FraudOps-Bench asked whether an LLM agent can emulate a fraud analyst's investigation process rather than just its output label, and evaluated that question with a selective-prediction framing that lets a system decline to decide rather than forcing a guess. The headline empirical result is a genuine tradeoff, not a clean win for either approach: under a correctly calibrated band, the agentic arm's peak accuracy claim holds up on fresh, never-touched data (96.1%, its best result on any split), but it earns that accuracy by escalating 83% of the queue, while a classical gradient-boosted-tree baseline remains the stronger practical performer across the full risk-coverage tradeoff. The more transferable result, we think, is not either arm's number but the calibration failure mode documented in Section 5: LLM confidence-score discretization interacting badly with a threshold selected on a small calibration set is a risk any selective-prediction pipeline built on LLM probability outputs should design around, independent of the underlying task.

Several deliberate scope cuts bound what this paper claims. No human fraud analyst was involved in this evaluation round – there is no human baseline, and the SOP driving the LLM arms' reasoning has not been validated by a practicing analyst (Section 9); both are natural next steps rather than oversights, and we do not claim this system matches or exceeds human analyst judgment. A second, structurally different dataset – BAF [12] is the strongest candidate, since it would extend the benchmark to account-opening fraud rather than the card-not-present setting studied here, a genuinely different investigation shape – was identified but deliberately deferred rather than integrated under this round's scope. Deeper architecture ablations beyond the single self-consistency check reported in Section 6 (supervisor-node necessity, tool-count sensitivity, in light of SOP-Bench's [8] finding that larger tool registries can hurt task success) and cross-model replication beyond the single Claude Sonnet 5 backend used throughout are both open. We see the calibration-failure diagnosis-and-fix methodology, more than any specific accuracy number reported here, as the part of this work most worth other selective-prediction practitioners carrying forward.

DATA AND CODE AVAILABILITY

The IEEE-CIS Fraud Detection dataset is a public Kaggle competition dataset [17] and is not redistributed in this repository, consistent with the competition's terms; researchers can obtain it directly from Kaggle. The processed, benchmark-specific case files derived from it – the dev/calibration/holdout_v1/holdout_v2 splits used for evaluation (data/processed/*.jsonl) – are included in the repository, since they contain only the small per-case evidence packets used by this benchmark's tools, not the raw competition data.

All code (agent implementation, tool definitions, SOP text, calibration/evaluation/statistics pipelines, and the frozen-methodology manifest described in Section 3.4) is available at github.com/pramegh-uikey/FraudOps-Bench.

ACKNOWLEDGMENTS

The authors used Claude (Anthropic) to draft portions of the Introduction, Related Work, Discussion, and Conclusion and Future Work sections, to identify and verify the prior-work citations in the References section, and to integrate existing experimental data (the Section 5.1a repeat-run variance table and the error-taxonomy findings referenced in Section 8) into the manuscript text. All experimental design, code, data collection, and statistical analysis were performed by the authors; all AI-assisted text and citations were reviewed and verified by the authors against the underlying source data (docs/methodology_log.md, docs/error_taxonomy.md, and outputs/holdout_v2/) before inclusion.

REFERENCES

[1] Association of Certified Fraud Examiners, *Occupational Fraud 2024: A Report to the Nations*. Austin, TX, USA: ACFE, 2024.
[2] UK Finance, *Half Year Fraud Report 2025*. London, U.K.: UK Finance, 2025.
[3] P. Grover, J. Xu, J. Tittelfitz, A. Cheng, Z. Li, J. Zablocki, J. Liu, and H. Zhou, "Fraud dataset benchmark and applications," *arXiv:2208.14417*, Aug. 2022.
[4] X. Tan, Y. Ma, and X. Zhang, "Understanding structured financial data with LLMs: A case study on fraud detection," *arXiv:2512.13040*, Dec. 2025.
[5] S. Shuster, E. Zaloof, A. Shabtai, and R. Puzis, "FAA framework: A large language model-based approach for credit card fraud investigations," *arXiv:2506.11635*, Jun. 2025.

[6] B. Wei, Y. S. Tay, H. Liu, J. Pan, K. Luo, Z. Zhu, and C. Jordan, "CORTEX: Collaborative LLM agents for high-stakes alert triage," *arXiv:2510.00311*, Sep. 2025.
[7] S. Jajodia, M. Sultana, S. Majumdar, A. Taylor, and G. Vandenberghe, "Before you hand over the wheel: Evaluating LLMs for security incident analysis," *arXiv:2603.06422*, Mar. 2026.
[8] S. Nandi *et al.*, "SOP-Bench: Complex industrial SOPs for evaluating LLM agents," *arXiv:2506.08119*, Jun. 2025.
[9] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv:2406.12045*, Jun. 2024.
[10] E. Levi and I. Kadar, "IntellAgent: A multi-agent framework for evaluating conversational AI systems," *arXiv:2501.11067*, Jan. 2025.
[11] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, PMLR vol. 267, 2025, pp. 48371–48392.
[12] S. Jesus, J. Pombal, D. Alves, A. Cruz, P. Saleiro, R. Ribeiro, J. Gama, and P. Bizarro, "Turning the tables: Biased, imbalanced, dynamic tabular datasets for ML evaluation," in *Proc. NeurIPS 2022 Datasets and Benchmarks Track*, *arXiv:2211.13358*.
[13] E. R. Altman, J. Blanusa, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Proc. NeurIPS 2023 Datasets and Benchmarks Track*, *arXiv:2306.16424*.
[14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework," *arXiv:2308.08155*, Aug. 2023.
[15] LangChain Inc., "LangGraph" [Software]. GitHub, 2024. [Online]. Available: https://github.com/langchain-ai/langgraph
[16] crewAI Inc., "CrewAI" [Software]. GitHub, 2023. [Online]. Available: https://github.com/crewAIInc/crewAI
[17] IEEE Computational Intelligence Society and Vesta Corporation, "IEEE-CIS Fraud Detection," Kaggle competition, 2019. [Online]. Available: https://www.kaggle.com/competitions/ieee-fraud-detection
[18] C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Trans. Inf. Theory*, vol. 16, no. 1, pp. 41–46, Jan. 1970, doi: 10.1109/TIT.1970.1054406.

PLACE PHOTO HERE

AYUSHI AMBILKAR received her degree in chemical engineering from the Indian Institute of Technology (IIT) Roorkee, India. She has approximately three years of industry experience as a data scientist, including work in the fraud detection and prevention domain at Accertify, a former subsidiary of American Express. Her research interests include applying large language model agents to fraud investigation workflows and selective-prediction evaluation methodology.

PLACE PHOTO HERE

PRAMEGH UIKEY received his degree in computer science engineering from the Indian Institute of Technology (IIT) Roorkee, India. He has approximately three years of industry experience as a data scientist. His research interests include tool-augmented large language model agents, benchmark design, and statistical evaluation of machine learning systems.